LONG SHORT-TERM MEMORY

Neural Computation 9(8):1735–1780, 1997

Sepp Hochreiter
Fakultät für Informatik
Technische Universität München
80290 München, Germany
hochreit@informatik.tu-muenchen.de
http://www7.informatik.tu-muenchen.de/~hochreit

Jürgen Schmidhuber
IDSIA
Corso Elvezia 36
6900 Lugano, Switzerland
juergen@idsia.ch
http://www.idsia.ch/~juergen

Abstract

Learning to store information over extended time intervals via recurrent backpropagation takes a very long time, mostly due to insufficient, decaying error back flow. We briefly review Hochreiter's 1991 analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called "Long Short-Term Memory" (LSTM). Truncating the gradient where this does not do harm, LSTM can learn to bridge minimal time lags in excess of 1000 discrete time steps by enforcing *constant* error flow through "constant error carrousels" within special units. Multiplicative gate units learn to open and close access to the constant error flow. LSTM is local in space and time; its computational complexity per time step and weight is $O(1)$. Our experiments with artificial data involve local, distributed, real-valued, and noisy pattern representations. In comparisons with RTRL, BPTT, Recurrent Cascade-Correlation, Elman nets, and Neural Sequence Chunking, LSTM leads to many more successful runs, and learns much faster. LSTM also solves complex, artificial long time lag tasks that have never been solved by previous recurrent network algorithms.

1 INTRODUCTION

Recurrent networks can in principle use their feedback connections to store representations of recent input events in form of activations ("short-term memory", as opposed to "long-term memory" embodied by slowly changing weights). This is potentially significant for many applications, including speech processing, non-Markovian control, and music composition (e.g., Mozer 1992). The most widely used algorithms for learning *what* to put in short-term memory, however, take too much time or do not work well at all, especially when minimal time lags between inputs and corresponding teacher signals are long. Although theoretically fascinating, existing methods do not provide clear *practical* advantages over, say, backprop in feedforward nets with limited time windows. This paper will review an analysis of the problem and suggest a remedy.

The problem. With conventional "Back-Propagation Through Time" (BPTT, e.g., Williams and Zipser 1992, Werbos 1988) or "Real-Time Recurrent Learning" (RTRL, e.g., Robinson and Fallside 1987), error signals "flowing backwards in time" tend to either (1) blow up or (2) vanish: the temporal evolution of the backpropagated error exponentially depends on the size of the weights (Hochreiter 1991). Case (1) may lead to oscillating weights, while in case (2) learning to bridge long time lags takes a prohibitive amount of time, or does not work at all (see section 3).

The remedy. This paper presents *"Long Short-Term Memory"* (LSTM), a novel recurrent network architecture in conjunction with an appropriate gradient-based learning algorithm. LSTM is designed to overcome these error back-flow problems. It can learn to bridge time intervals in excess of 1000 steps even in case of noisy, incompressible input sequences, without loss of short time lag capabilities. This is achieved by an efficient, gradient-based algorithm for an architecture

where

$$net_{out_j}(t) = \sum_u w_{out_j u} y^u(t-1),$$

and

$$net_{in_j}(t) = \sum_u w_{in_j u} y^u(t-1).$$

We also have

$$net_{c_j}(t) = \sum_u w_{c_j u} y^u(t-1).$$

The summation indices u may stand for input units, gate units, memory cells, or even conventional hidden units if there are any (see also paragraph on "network topology" below). All these different types of units may convey useful information about the current state of the net. For instance, an input gate (output gate) may use inputs from other memory cells to decide whether to store (access) certain information in its memory cell. There even may be recurrent self-connections like $w_{c_j c_j}$. It is up to the user to define the network topology. See Figure 2 for an example.

At time t, c_j's output $y^{c_j}(t)$ is computed as

$$y^{c_j}(t) = y^{out_j}(t) h(s_{c_j}(t)),$$

where the "internal state" $s_{c_j}(t)$ is

$$s_{c_j}(0) = 0, s_{c_j}(t) = s_{c_j}(t-1) + y^{in_j}(t) g\left(net_{c_j}(t)\right) \text{ for } t > 0.$$

The differentiable function g squashes net_{c_j}; the differentiable function h scales memory cell outputs computed from the internal state s_{c_j}.

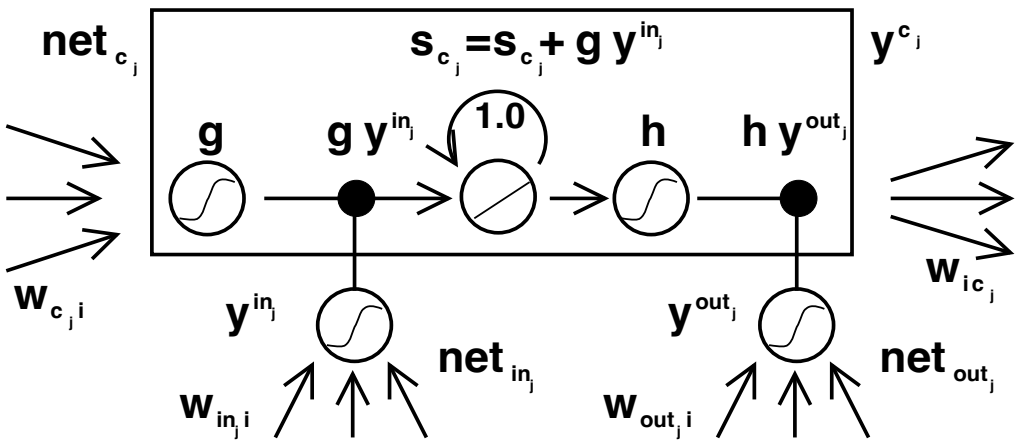

Figure 1: *Architecture of memory cell c_j (the box) and its gate units in_j, out_j. The self-recurrent connection (with weight 1.0) indicates feedback with a delay of 1 time step. It builds the basis of the "constant error carrousel" CEC. The gate units open and close access to CEC. See text and appendix A.1 for details.*

Why gate units? To avoid input weight conflicts, in_j controls the error flow to memory cell c_j's input connections $w_{c_j i}$. To circumvent c_j's output weight conflicts, out_j controls the error flow from unit j's output connections. In other words, the net can use in_j to decide when to keep or override information in memory cell c_j, and out_j to decide when to access memory cell c_j and when to prevent other units from being perturbed by c_j (see Figure 1).

Error signals trapped within a memory cell's CEC *cannot* change – but different error signals flowing into the cell (at different times) via its output gate may get superimposed. The output gate will have to learn *which* errors to trap in its CEC, by appropriately scaling them. The input

gate will have to learn when to release errors, again by appropriately scaling them. Essentially, the multiplicative gate units open and close access to constant error flow through CEC.

Distributed output representations typically do require output gates. Not always are both gate types necessary, though — one may be sufficient. For instance, in Experiments 2a and 2b in Section 5, it will be possible to use input gates only. In fact, output gates are not required in case of local output encoding — preventing memory cells from perturbing already learned outputs can be done by simply setting the corresponding weights to zero. Even in this case, however, output gates can be beneficial: they prevent the net's attempts at storing long time lag memories (which are usually hard to learn) from perturbing activations representing easily learnable short time lag memories. (This will prove quite useful in Experiment 1, for instance.)

Network topology. We use networks with one input layer, one hidden layer, and one output layer. The (fully) self-connected hidden layer contains memory cells and corresponding gate units (for convenience, we refer to both memory cells and gate units as being located in the hidden layer). The hidden layer may also contain "conventional" hidden units providing inputs to gate units and memory cells. All units (except for gate units) in all layers have directed connections (serve as inputs) to all units in the layer above (or to all higher layers – Experiments 2a and 2b).

Memory cell blocks. S memory cells sharing the same input gate and the same output gate form a structure called a "memory cell block of size S". Memory cell blocks facilitate information storage — as with conventional neural nets, it is not so easy to code a distributed input within a single cell. Since each memory cell block has as many gate units as a single memory cell (namely two), the block architecture can be even slightly more efficient (see paragraph "computational complexity"). A memory cell block of size 1 is just a simple memory cell. In the experiments (Section 5), we will use memory cell blocks of various sizes.

Learning. We use a variant of RTRL (e.g., Robinson and Fallside 1987) which properly takes into account the altered, multiplicative dynamics caused by input and output gates. However, to ensure non-decaying error backprop through internal states of memory cells, as with truncated BPTT (e.g., Williams and Peng 1990), errors arriving at "memory cell net inputs" (for cell c_j, this includes net_{c_j}, net_{in_j}, net_{out_j}) do not get propagated back further in time (although they *do* serve to change the incoming weights). Only within[2] memory cells, errors are propagated back through previous internal states s_{c_j}. To visualize this: once an error signal arrives at a memory cell output, it gets scaled by output gate activation and h'. Then it is within the memory cell's CEC, where it can flow back indefinitely without ever being scaled. Only when it leaves the memory cell through the input gate and g, it is scaled once more by input gate activation and g'. It then serves to change the incoming weights before it is truncated (see appendix for explicit formulae).

Computational complexity. As with Mozer's focused recurrent backprop algorithm (Mozer 1989), only the derivatives $\frac{\partial s_{c_j}}{\partial w_{il}}$ need to be stored and updated. Hence the LSTM algorithm is very efficient, with an excellent update complexity of $O(W)$, where W the number of weights (see details in appendix A.1). Hence, LSTM and BPTT for fully recurrent nets have the same update complexity per time step (while RTRL's is much worse). Unlike full BPTT, however, LSTM is *local in space and time*[3]: there is no need to store activation values observed during sequence processing in a stack with potentially unlimited size.

Abuse problem and solutions. In the beginning of the learning phase, error reduction may be possible without storing information over time. The network will thus tend to abuse memory cells, e.g., as bias cells (i.e., it might make their activations constant and use the outgoing connections as adaptive thresholds for other units). The potential difficulty is: it may take a long time to release abused memory cells and make them available for further learning. A similar "abuse problem" appears if two memory cells store the same (redundant) information. There are at least two solutions to the abuse problem: *(1) Sequential network construction* (e.g., Fahlman 1991): a memory cell and the corresponding gate units are added to the network whenever the

[2]For intra-cellular backprop in a quite different context see also Doya and Yoshizawa (1989).

[3]Following Schmidhuber (1989), we say that a recurrent net algorithm is *local in space* if the update complexity per time step and weight does not depend on network size. We say that a method is *local in time* if its storage requirements do not depend on input sequence length. For instance, RTRL is local in time but not in space. BPTT is local in space but not in time.

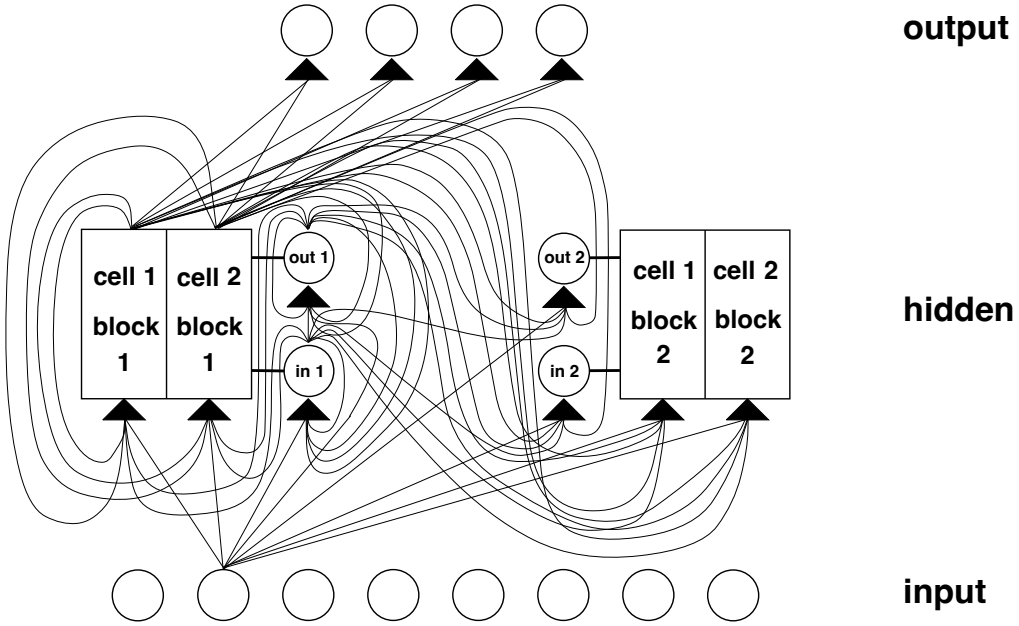

Figure 2: *Example of a net with 8 input units, 4 output units, and 2 memory cell blocks of size 2.* $in1$ *marks the input gate,* $out1$ *marks the output gate, and* $cell1/block1$ *marks the first memory cell of block 1.* $cell1/block1$*'s architecture is identical to the one in Figure 1, with gate units* $in1$ *and* $out1$ *(note that by rotating Figure 1 by 90 degrees anti-clockwise, it will match with the corresponding parts of Figure 1). The example assumes dense connectivity: each gate unit and each memory cell see all non-output units. For simplicity, however, outgoing weights of only one type of unit are shown for each layer. With the efficient, truncated update rule, error flows only through connections to output units, and through fixed self-connections within cell blocks (not shown here — see Figure 1). Error flow is truncated once it "wants" to leave memory cells or gate units. Therefore, no connection shown above serves to propagate error back to the unit from which the connection originates (except for connections to output units), although the connections themselves are modifiable. That's why the truncated LSTM algorithm is so efficient, despite its ability to bridge very long time lags. See text and appendix A.1 for details. Figure 2 actually shows the architecture used for Experiment 6a — only the bias of the non-input units is omitted.*

error stops decreasing (see Experiment 2 in Section 5). *(2) Output gate bias:* each output gate gets a negative initial bias, to push initial memory cell activations towards zero. Memory cells with more negative bias automatically get "allocated" later (see Experiments 1, 3, 4, 5, 6 in Section 5).

Internal state drift and remedies. If memory cell c_j's inputs are mostly positive or mostly negative, then its internal state s_j will tend to drift away over time. This is potentially dangerous, for the $h'(s_j)$ will then adopt very small values, and the gradient will vanish. One way to circumvent this problem is to choose an appropriate function h. But $h(x) = x$, for instance, has the disadvantage of unrestricted memory cell output range. Our simple but effective way of solving drift problems at the beginning of learning is to initially bias the input gate in_j towards zero. Although there is a tradeoff between the magnitudes of $h'(s_j)$ on the one hand and of y^{in_j} and f'_{in_j} on the other, the potential negative effect of input gate bias is negligible compared to the one of the drifting effect. With logistic sigmoid activation functions, there appears to be no need for fine-tuning the initial bias, as confirmed by Experiments 4 and 5 in Section 5.4.

5 EXPERIMENTS

Introduction. Which tasks are appropriate to demonstrate the quality of a novel long time lag

method	hidden units	# weights	learning rate	% of success	success after
RTRL	3	≈ 170	0.05	"some fraction"	173,000
RTRL	12	≈ 494	0.1	"some fraction"	25,000
ELM	15	≈ 435		0	>200,000
RCC	7-9	$\approx$ 119-198		50	182,000
LSTM	4 blocks, size 1	264	0.1	100	39,740
LSTM	3 blocks, size 2	276	0.1	100	21,730
LSTM	3 blocks, size 2	276	0.2	97	14,060
LSTM	4 blocks, size 1	264	0.5	97	9,500
LSTM	3 blocks, size 2	276	0.5	100	8,440

Table 1: *EXPERIMENT 1: Embedded Reber grammar: percentage of successful trials and number of sequence presentations until success for RTRL (results taken from Smith and Zipser 1989), "Elman net trained by Elman's procedure" (results taken from Cleeremans et al. 1989), "Recurrent Cascade-Correlation" (results taken from Fahlman 1991) and our new approach (LSTM). Weight numbers in the first 4 rows are estimates — the corresponding papers do not provide all the technical details. Only LSTM almost always learns to solve the task (only two failures out of 150 trials). Even when we ignore the unsuccessful trials of the other approaches, LSTM learns much faster (the number of required training examples in the bottom row varies between 3,800 and 24,100).*

tations. A successful run is one that fulfills the following criterion: after training, during 10,000 successive, randomly chosen input sequences, the maximal absolute error of all output units is always below 0.25.

Architectures. RTRL: one self-recurrent hidden unit, $p+1$ non-recurrent output units. Each layer has connections from all layers below. All units use the logistic activation function sigmoid in [0,1].

BPTT: same architecture as the one trained by RTRL.

CH: both net architectures like RTRL's, but one has an additional output for predicting the hidden unit of the other one (see Schmidhuber 1992b for details).

LSTM: like with RTRL, but the hidden unit is replaced by a memory cell and an input gate (no output gate required). g is the logistic sigmoid, and h is the identity function $h : h(x) = x, \forall x$. Memory cell and input gate are added once the error has stopped decreasing (see abuse problem: solution (1) in Section 4).

Results. Using RTRL and a short 4 time step delay ($p = 4$), $\frac{7}{9}$ of all trials were successful. *No trial was successful with* $p = 10$. With *long* time lags, only the neural sequence chunker and LSTM achieved successful trials, while BPTT and RTRL failed. With $p = 100$, the 2-net sequence chunker solved the task in only $\frac{1}{3}$ of all trials. LSTM, however, always learned to solve the task. Comparing successful trials only, LSTM learned much faster. See Table 2 for details. It should be mentioned, however, that a *hierarchical* chunker can also always quickly solve this task (Schmidhuber 1992c, 1993).

Task 2b: no local regularities. With the task above, the chunker sometimes learns to correctly predict the final element, but only because of predictable local regularities in the input stream that allow for compressing the sequence. In an additional, more difficult task (involving many more different possible sequences), we remove compressibility by replacing the deterministic subsequence $(a_1, a_2, \ldots, a_{p-1})$ by a *random* subsequence (of length $p-1$) over the alphabet $a_1, a_2, \ldots, a_{p-1}$. We obtain 2 classes (two sets of sequences) $\{(y, a_{i_1}, a_{i_2}, \ldots, a_{i_{p-1}}, y) \mid 1 \leq i_1, i_2, \ldots, i_{p-1} \leq p-1\}$ and $\{(x, a_{i_1}, a_{i_2}, \ldots, a_{i_{p-1}}, x) \mid 1 \leq i_1, i_2, \ldots, i_{p-1} \leq p-1\}$. Again, every next sequence element has to be predicted. The only totally predictable targets, however, are x and y, which occur at sequence ends. Training exemplars are chosen randomly from the 2 classes. Architectures and parameters are the same as in Experiment 2a. A successful run is one that fulfills the following criterion: after training, during 10,000 successive, randomly chosen input

Method	Delay p	Learning rate	# weights	% Successful trials	Success after
RTRL	4	1.0	36	78	1,043,000
RTRL	4	4.0	36	56	892,000
RTRL	4	10.0	36	22	254,000
RTRL	10	1.0-10.0	144	0	> 5,000,000
RTRL	100	1.0-10.0	10404	0	> 5,000,000
BPTT	100	1.0-10.0	10404	0	> 5,000,000
CH	100	1.0	10506	33	32,400
LSTM	100	1.0	10504	100	5,040

Table 2: *Task 2a: Percentage of successful trials and number of training sequences until success, for "Real-Time Recurrent Learning" (RTRL), "Back-Propagation Through Time" (BPTT), neural sequence chunking (CH), and the new method (LSTM). Table entries refer to means of 18 trials. With 100 time step delays, only CH and LSTM achieve successful trials. Even when we ignore the unsuccessful trials of the other approaches, LSTM learns much faster.*

sequences, the maximal absolute error of all output units is below 0.25 at sequence end.

Results. As expected, the chunker failed to solve this task (so did BPTT and RTRL, of course). LSTM, however, was always successful. On average (mean of 18 trials), success for $p = 100$ was achieved after 5,680 sequence presentations. This demonstrates that LSTM does not require sequence regularities to work well.

Task 2c: very long time lags — no local regularities. This is the most difficult task in this subsection. To our knowledge no other recurrent net algorithm can solve it. Now there are $p+4$ possible input symbols denoted $a_1, ..., a_{p-1}, a_p, a_{p+1} = e, a_{p+2} = b, a_{p+3} = x, a_{p+4} = y$. $a_1, ..., a_p$ are also called *"distractor symbols"*. Again, a_i is locally represented by the $p+4$-dimensional vector whose ith component is 1 (all other components are 0). A net with $p+4$ input units and 2 output units sequentially observes input symbol sequences, one at a time. Training sequences are randomly chosen from the union of two very similar subsets of sequences: $\{(b, y, a_{i_1}, a_{i_2}, \ldots, a_{i_{q+k}}, e, y) \mid 1 \leq i_1, i_2, \ldots, i_{q+k} \leq q\}$ and $\{(b, x, a_{i_1}, a_{i_2}, \ldots, a_{i_{q+k}}, e, x) \mid 1 \leq i_1, i_2, \ldots, i_{q+k} \leq q\}$. To produce a training sequence, we (1) randomly generate a sequence prefix of length $q + 2$, (2) randomly generate a sequence suffix of additional elements ($\neq b, e, x, y$) with probability $\frac{9}{10}$ or, alternatively, an e with probability $\frac{1}{10}$. In the latter case, we (3) conclude the sequence with x or y, depending on the second element. For a given k, this leads to a uniform distribution on the possible sequences with length $q + k + 4$. The minimal sequence length is $q + 4$; the expected length is

$$4 + \sum_{k=0}^{\infty} \frac{1}{10} (\frac{9}{10})^k (q + k) = q + 14.$$

The expected number of occurrences of element $a_i, 1 \leq i \leq p$, in a sequence is $\frac{q+10}{p} \approx \frac{q}{p}$. The goal is to predict the last symbol, which always occurs after the "trigger symbol" e. Error signals are generated only at sequence ends. To predict the final element, the net has to learn to store a representation of the second element for at least $q + 1$ time steps (until it sees the trigger symbol e). Success is defined as "prediction error (for final sequence element) of both output units always below 0.2, for 10,000 successive, randomly chosen input sequences".

Architecture/Learning. The net has $p + 4$ input units and 2 output units. Weights are initialized in [-0.2,0.2]. To avoid too much learning time variance due to different weight initializations, the hidden layer gets two memory cells (two cell blocks of size 1 — although one would be sufficient). There are no other hidden units. The output layer receives connections only from memory cells. Memory cells and gate units receive connections from input units, memory cells and gate units (i.e., the hidden layer is fully connected). No bias weights are used. h and g are logistic sigmoids with output ranges $[-1, 1]$ and $[-2, 2]$, respectively. The learning rate is 0.01.